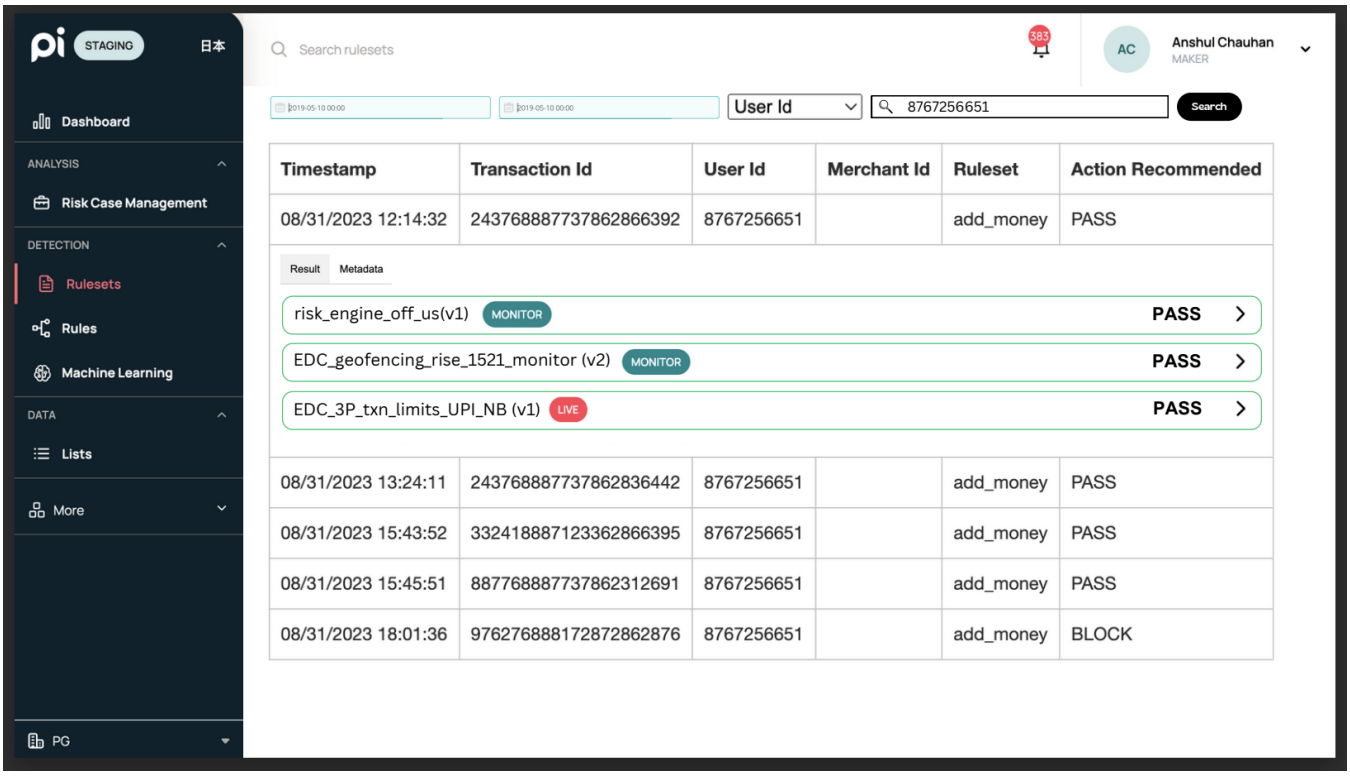
Rule Debugger (RC Lookup)

Objective :- The objective of this feature is to create a unified and centralised platform that enables users to efficiently access and retrieve comprehensive information related to fraud check transactions. By consolidating all relevant data in a single location, this feature aims to enhance user productivity, reduce the time spent searching for information, and improve decision-making in fraud detection and prevention processes.

Visualisation :- Visualisation



Note: The image provided is for illustrative purposes only and serves as a sample representation of the potential visualisation. The actual visualisation may vary in terms of design, layout, colours, and content. The final product will be tailored to meet the specific requirements and aesthetics of the project. Please consider this image as a conceptual preview and anticipate that the actual visual representation may differ.

Implementation :-

1. **Backend Service with Data Aggregation ('Lumique') :** Develop a backend service responsible for aggregating and serving fraud check transaction information to the frontend.
   Provide summary of audit log for given transaction including information like
   a. rule result information (PASS/BLOCK/INCONCLUSIVE)
   b. DIT/TIMEOUT causing rule.
2. **UI Interface Layer:** Design an intuitive and user-friendly web interface to provide a centralised view of fraud check transaction data.

Use cases :-

1. The system retrieves all* transactions associated with the specified UserId within the provided timeframe.
2. The system fetches all* transactions related to the given MerchantId within the specified timeframe.
3. The system fetches all* transactions related to the given BuyerId within the specified timeframe.
4. The system retrieves transaction details that correspond to the provided TransactionId and fall within the specified timeframe.
5. The system retrieves transaction details that correspond to the provided OrderId and fall within the specified timeframe.

* The number of results will be limited based on the configuration defined in "Lumique". For instance, if the configured limit is set to 15, the system will retrieve the first 15 transactions that match the search criteria, ordered in ascending order of timestamp.

Note :- The system will be capable of fetching data for a maximum duration of 90 days. Transactions older than this period will be subject to a separate process for data retrieval.

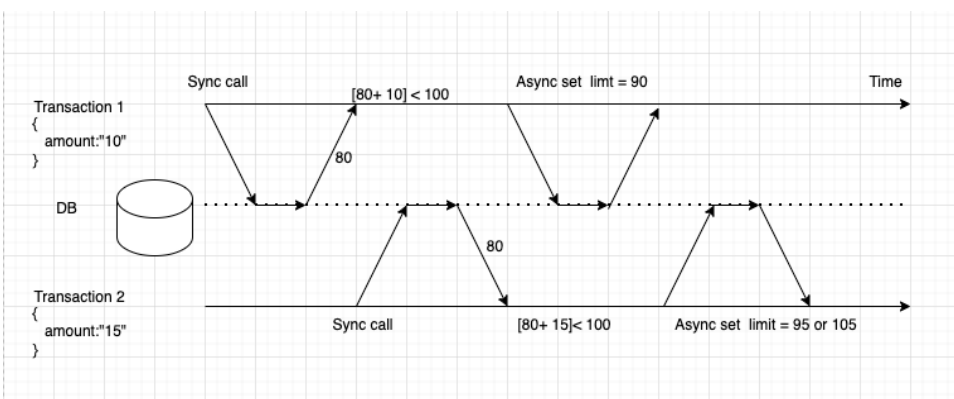
Data Source :-

1. **Audit Logs Retrieval :-**

Source	Pros	Cons	Note
ElasticSearch	Existing integration already setup. Easy to fetch data programmatically.Fast query result	Not cost effective. Can be taken down in future . Limited past data retention	
Starburst over S3	Starburst Enterprise already setup by central team. Audit logs can be queried in their own table . Longer data retention . Provides sql type query language to query data	Table configuration and setup maybe required.	As of now central data team has disallowed the usage of api to query data. Hence not possible to query apart from starburst ui or looker.
Athena over S3	Tighter control on data to put into tables generated from logs. Additional infrastructure not needed to run query engine. Managed by AWS.	Will require additional cost of running athena engine . Per query charge model.	
Presto Cluster	Robust Query Engine that can be run over s3 logs and many more data sources. Libraries and SDK easily available to integrate programmatically.	Will require additional cost and development effort to setup presto cluster	

Cost Analysis **Data Source Cost Comparison**

2. **Ruleset Information Retrieval :-**
    a. **Approach 1: Audit Log and API Integration :** Retrieve ruleset information by analysing audit logs to identify the evaluation type. Use this evaluation type to trigger API calls to the Rule Admin service. These APIs provide comprehensive details about rulesets and rules associated with the evaluation type.
    b. **Approach 2: Local Memory Snapshot :** Create and maintain a local memory snapshot of ruleset information within the service. This snapshot will store cached details about rulesets and rules. By using the snapshot, you can avoid latency caused by frequent API calls. Refresh the snapshot daily and allow manual refreshing through a trigger.

Rule limit breach detection (Phase 2):-

1. Rule Limit breach detection due to velocity not captured in time.
   Given a transaction, check if limit breach happened in and around the transaction.
   - fetch few transactions around the given transaction for a user/merchantid.
   - Check if two consecutive transactions share same result for a given query but previous transaction has velocity field in payload which means this particular velocity field should have affected the attribute used in current transaction's query result